

An Analysis of Lightweight Convolutional Neural Network Models for Image Classification Task on Edge Device

Muhammad Abbas Abu Talib
Electrical Engineering Studies,
College of Engineering,
Universiti Teknologi MARA,
Cawangan Pulau Pinang,
Permatang Pauh, Pulau Pinang,
Malaysia.

2024740247@student.uitm.edu.my

Samsul Setumin
Electrical Engineering Studies,
College of Engineering,
Universiti Teknologi MARA,
Cawangan Pulau Pinang,
Permatang Pauh, Pulau Pinang,
Malaysia.

samsuls@uitm.edu.my

Adi Izhar Che Ani
Electrical Engineering Studies,
College of Engineering,
Universiti Teknologi MARA,
Cawangan Pulau Pinang,
Permatang Pauh, Pulau Pinang,
Malaysia.

adiizhar@uitm.edu.my

Siti Juliana Abu Bakar
Electrical Engineering Studies,
College of Engineering,
Universiti Teknologi MARA,
Cawangan Pulau Pinang,
Permatang Pauh, Pulau Pinang,
Malaysia.

sitijuliana@uitm.edu.my

Abstract — A lightweight Convolutional Neural Network (CNN) has become one of the major studies in machine learning field to optimize its potential for employing it on the resource-constrained devices. However, a benchmark for fair comparison is still missing and thus, this paper aims to analyze the recent studies regarding the lightweight CNN models for image classification tasks on edge devices. For that, the study objective is to investigate the current lightweight CNN models that are best for image classification task on edge device while the second objective is to evaluate and determine the most feasible model to be deployed on edge device (i.e., NVIDIA Jetson Nano). As methodology, several models were investigated and selected including a few variants from MobileNet, ShuffleNet, and VGG16 for performance analysis. The approach followed by transfer learning with training and validation were done on a computer and their inference were deployed and ran on Jetson Nano. In addition, the dataset used for the transfer learning is obtained from a dataset of image classification for different classes of leaf diseases for a groundnut plant. Then, performance evaluation according to training, validation, and inference results shows that the MobileNet variants were on the lower performance compared to VGG16 and ShuffleNetV2 variants. In addition, ShuffleNetV2 has better performance with 98.06% classification accuracy, 9.41 ms of inference time and 106.76 FPS. Thus, for future research, ShuffleNetV2 can be further optimize and modify so that it can be better support and generalization for various edge applications.

Keywords — *lightweight CNN, edge device, image classification, resource-constrained, Jetson Nano.*

I. INTRODUCTION

Researchers these days are primarily focused on the advancement of Artificial Intelligence (AI) technology in order to enhance society's quality of life and facilitate the industrial revolution. Although this discipline was introduced back in the 1950s, it has gone through rapid development in the past

decades, which has covered both inside and outside of the computer science field [1], [2]. It can be seen that many technologies and non-technology-based journals have published articles related to AI [1], [2]. AI has progressed from simple rule-based systems to more complicated algorithms that can make autonomous decisions and solve problems. The primary idea underlying AI is to develop systems capable of doing activities that would normally need human intellect, such as visual perception, speech recognition, decision-making, and language translation [1], [2].

Furthermore, Machine Learning (ML) is one of the most common subfields in AI, where it takes a different approach from a classical programming method. So, instead of using an algorithm for a specific problem or function, ML use a certain dataset for its algorithm to learn, predict, and decide the outcome [1], [2]. Primarily, ML is typically categorized into four major types such as supervised learning, which involves training models on labelled data, unsupervised learning, which involves searching for patterns in unlabelled data, semi supervised learning, which includes both supervised and unsupervised learning, and reinforcement learning, which teaches models to make decisions based on trial and error [1], [2]. In addition, the application of ML is commonly divided into object classification or regression (i.e., prediction). Some typical examples of algorithms in ML include artificial neural network, decision trees, linear regression, and support vector machine [1], [2].

Moreover, Convolutional Neural Network (CNN) is one of ML's artificial neural network algorithms that is specialized for image-based tasks [1], [3], [4], [5], [6]. In other words, CNN is fundamental in many computer vision tasks such as image detection, recognition, classification, regression, and segmentation. CNN is made up of three main layers, including convolutional, pooling, and fully connected layers [1], [3], [4],

[5], [6]. For the basic of CNN architecture, first, convolutional layers apply filters to incoming data, capturing spatial hierarchies and local patterns that are necessary for applications such as image identification [1], [3], [4], [5], [6]. Second, pooling layers lowers the dimensionality of the data, increasing computing efficiency and resilience [1], [3], [4], [5], [6]. Third, fully connected layers make high-level decisions based on the extracted characteristics [1], [3], [4], [5], [6]. Simply put, the first two main layers perform feature extraction from the input data and the third main layer maps the extracted features to decide or predict the output data [1], [3], [4], [5], [6].

Nevertheless, the development of CNN's applications usually involves with big data which relies heavily on cloud infrastructure and resources for high computation complexity, memory and load power consumption [4], [5], [6], [7], [8]. In recent years, the widespread use of cloud computing in many fields of CNN's applications has raised some concern regarding strict latency requirements, strained network capacity, as well as privacy and security issues [4], [5], [6], [7], [8]. Ultimately, in order to overcome these problems and optimize CNN's applications, an increasing demand for deploying deep learning models directly onto edge devices to enable real-time inference and decision-making has been introduced. Edge computing includes processing data at or near the source of data creation which is called edge devices, such as IoT devices, smartphones, or sensors, rather than using a centralized cloud infrastructure.

Edge computing can become a breakthrough solution in replacing certain applications of AI, where strict latency requirements, secured data privacy, and low bandwidth usage are needed. However, optimizing their efficiency for edge deployment poses a critical challenge due to limited computational resources and power constraints. According to the research in [9], several research challenges in achieving stable, robust, and practical usage of edge computing require non-stop effort and continuous study from the perspectives of model training, model inference, application enhancement, as well as hardware and software optimization. In addition, the benchmark for a fair comparison and generalization is still missing since the applications of those existing models vary from each other [8], [9], [10], [11], [12]. For example, the percentage of accuracy produced from a model trained with ImageNet dataset compared to the same model trained with MNIST dataset will be different because of the complexity of the image differ from one another, as stated in [12].

By considering that the subject is still in the early stage of development, thus, this work aims to analyse CNN models tailored specifically for deployment on an edge device for image classification tasks. Furthermore, as many researchers have employed edge computing to gain various benefits, most study in this subject have used different method of optimization to produce their lightweight CNN for edge device deployment. Thus, a thorough investigation needs to be conducted to grasp an understanding of a few recently produced of those state-of-the-art approach. In short, the first objective of this study is to investigate the current lightweight CNN models that best for image classification task on edge device and the second objective is to evaluate and determine the most feasible model to be deployed on edge device.

II. METHODOLOGY

In this methodology section, the whole process of executing this study will be explained as summarizes in Fig. 1.

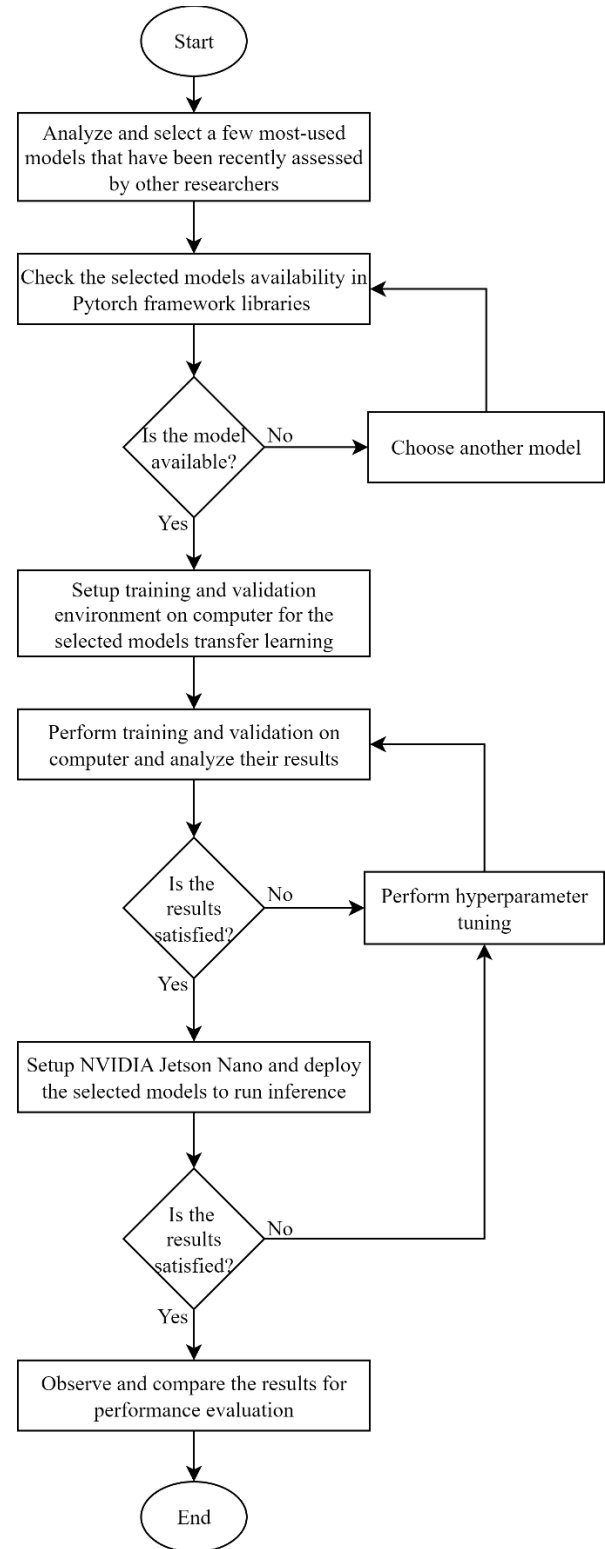


Fig. 1. Methodology flowchart of this study.

Initially, the full approach of this work starts with analyzing and selecting a few most used that have recently been assessed by other researchers. Followed by that, the availability of the selected models will be checked in Pytorch framework libraries as Pytorch is an open source application for machine learning and deep learning library. After obtaining all the selected models, a training and validation environment will be setup on a computer hardware. Subsequently, the selected models will be trained and validated on the computer. Once all the training and validation result are satisfied, a testing environment on an edge device will be setup. Then, the selected models will be deployed to run an inference on the edge device. Last but not least, all the result will be compiled for performance evaluation.

A. Analysis and selection of recent lightweight CNN models

Firstly, Fig. 4 summarizes most of the current research on lightweight CNN applications. It can be seen that there were several similar networks that were being used as their approaches, such as MobileNet with 9 (19.1%), ShuffleNet with 4 (8.5%), VGG-16 with 4 (8.5%), CondenseNet with 3 (6.4%), etc., respectively in terms of number of their usage. Meanwhile, the Others with 21 (44.7%), represent the number of different models with only one usage. Primarily, the top three models that have been highly used such as MobileNet, ShuffleNet, and VGG16, were selected for further analysis.

MobileNet is known for its balance of accuracy and computing efficiency, making it suitable for mobile and embedded applications. Moreover, ShuffleNet is also considered for this research as it also has been moderately used for several researchers. ShuffleNet is define from its architecture key feature as it uses group convolutions and channel shuffling to minimize computation while preserving accuracy. Other than that, VGG16 is recognized as selected model for it also has been similarly used as ShuffleNet. VGG16 is known about its compact architecture, which employs a conventional deep stacked convolutional structure to attain high accuracy for a slower learning rate.

Aside from that, the availability of the selected models is also checked from an open-source ML library such as Pytorch in order to ensure that these models can be utilized and developed with a fully featured framework. For that, several variances of model from MobileNet, such as MobileNetV2 [13] and MobileNetV3 [14] are furthered selected. For MobileNetV3, there are two more variation which are MobileNetV3 - Small model and MobileNetV3 - Large model. On the other hand, the ShuffleNetV2 [15] is chosen instead of ShuffleNet, with four more variation such as ShuffleNetV2 x0.5, ShuffleNetV2 x1.0, ShuffleNetV2 x1.5, and ShuffleNetV2 x2.0. With that, the total number of the selected models for further analysis is 8 including VGG16 [16].

B. Training and validation environments setup on computer

After selecting some of the recent lightweight CNN models, the training and validation environments on computer hardware will be configured. This subprocess is included after consideration that a lightweight CNN model does not necessarily need to be trained on an edge device as long as its model inference can be run properly for real life image classification task. Moreover, running model training can add

more strain on the computation ability of the edge device as it increases its memory usage instead of just running a model inference.

So, first of all, a dataset for image classification task is determined. In this step, a fixed dataset is used on all the selected for training and validation. This dataset was chosen in order to evaluate the feasibility of the selected models on edge device which can be utilize as one of the edge computing solutions in real-world application. Specifically, this dataset is a dataset of image classification for different classes or types of leaf diseases for a groundnut plant [17]. The classes of each type of leaf diseases are divided into 6 classes such as early leaf spot, 1065 images of early rust, 1491 images of late leaf spot, 1255 images of nutrition deficiency, 1315 images of rust, and including 1462 images of healthy leaf class. All in all, the total number of sample images for training is 7910 for all classes combined. For data augmentation, 30% of each class sample images are used for validation. Furthermore, all of the images for both training and validation are being randomly organized in different directions and different backgrounds. However, image size and channel remain fixed for training and validation which are 256x256x3.

After that, the setting for software tools and computer hardware are configured. First, the application environment is set within Anaconda software in order to manage the necessary libraries' dependency. Next, Python is installed as the programming language and CUDA API is also installed to enable the NVIDIA GeForce GTX 1060 GPU 6GB as the computing tool. Then, for the model framework, Pytorch is used to retrieve all the selected pre-trained models that were trained using ImageNet1K dataset and their libraries for transfer learning. Additionally, Pytorch framework was chosen as it provides more flexibility (i.e., GPU support for Windows OS and custom libraries) for research purpose compared to TensorFlow or Keras. Also, this system is executed on a Windows 11 Pro OS computer platform.

Last but not least, hyperparameter setting for each of the selected models are also configured as the final step before executing each of the training and validation. In order to obtain more reliable comparisons for this analysis, most of the hyperparameter settings for the models are kept at default and fixed value to minimize their variables. However, except for VGG16, its learning rate has to be much smaller than the others in order to avoid loss function break and keep its weight stabilized due to the absence of normalization in its architecture [16]. Table I summarizes all the hyperparameter settings. Also, all the selected models were trained and validated three times with similar hyperparameter settings in order to obtain a robust average result.

C. Inference environments setup on edge device

After completing the training and validation on computer, the setup on edge device is configured to deploy and run inference. For that, first, the NVIDIA Jetson Nano is selected as the edge computing platform. This device comes with Jetpack SDK as its firmware which includes all the necessary tools for AI development such as CUDA, TensorRT, ONNX, and others. Also, Linux Ubuntu is used as the OS for this device and the application environment for this work is configured within a

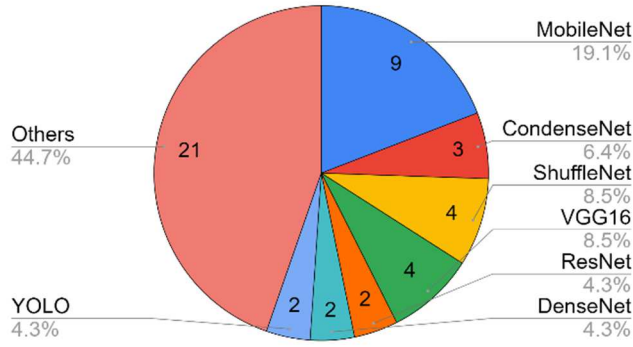


Fig. 2. Several common lightweight CNN models.

docker container. Other than that, Python is also used as the programming language including the inference script. Similar to the training process, all the selected models were also inferred three times on Jetson Nano to obtain the average results.

D. Result collection and performance evaluations

For this part, each data for the results of training, validation, and inference are collected and analyzed. Specifically, the main evaluation metric criteria taking into account are the train accuracy and loss, validation accuracy and loss, and classification accuracy of each inference of the selected models.

For training and validation, accuracy is the most basic evaluation metric in ML for both training and testing. Unlike accuracy, loss is often used only for training and validation. In short, accuracy is calculated simply by taking the number of correct predictions over the number of total samples. However, loss is the summation of errors of the loss function per epoch from the training and validation of the model. Ideally, accuracy should keep increasing, while loss keep decreasing as the training goes on.

For testing and inference, classification accuracy is basically the overall accuracy of the model. By taking the total number of correct predictions divide by the total number of sample images and multiply by 100, the percentage accuracy of the models is obtained as shown in (1). For inference time, the time taken was captured before and after each inference per image as shown in (2). Also, for frames per second (FPS), 1 second is divided by inference time as shown in (3). Both these inference time and FPS can show the speed of the model with regards to its feasibility as edge computing application.

$$Accuracy (\%) = \left(\frac{\sum \text{Correct Predictions}}{\sum \text{Sample Images}} \right) \times 100 \quad (1)$$

$$Inference Time (ms) = t_{stop} (ms) - t_{start} (ms) \quad (2)$$

where, t = time

$$FPS = \frac{1000ms}{Inference Time (ms)} \quad (3)$$

III. RESULT AND DISCUSSION

In this section, all the data obtained for the result of training, validation, and testing or inference will be discussed.

Moreover, the result and discussion will be separated into two subsections. First, the performance of training, validation, and inference for each selected model will be tabulated and discussed. Then, their overall performance will be examined in the second subsection.

TABLE I. TRAINING HYPERPARAMETER SETTING FOR THE SELECTED MODEL

Model	Epoch	Batch Size	Learning Rate	Momentum	Weight Decay
MobileNet V2	35	8	0.1	0.9	1e-4
MobileNet V3-Small	35	8	0.1	0.9	1e-4
MobileNet V3-Large	35	8	0.1	0.9	1e-4
ShuffleNet V2x0.5	35	8	0.1	0.9	1e-4
ShuffleNet V2x1.0	35	8	0.1	0.9	1e-4
ShuffleNet V2x1.5	35	8	0.1	0.9	1e-4
ShuffleNet V2x2.0	35	8	0.1	0.9	1e-4
VGG-16	35	8	0.005	0.9	1e-4

A. Training, validation, and inference result

Looking into Table II, it summarizes all each of the selected models' average performance for training and validation using transfer learning. The table includes all the data specifically during the initial and final epoch for the particular process. In terms of final epoch, it can be seen that VGG16 has the lowest loss and highest accuracy across all four metrics. Meanwhile, MobileNetV2 has higher increasing in train accuracy and validation accuracy, while MobileNetV3 has higher decreasing in train loss for the large variant. Aside from that, all the ShuffleNetV2 models have average performance respectively.

For further analysis, Table III tabulates all the classification accuracy, inference time, and FPS during inference in Jetson Nano. Here, it seems that the highest accuracy is obtained by ShuffleNetV2 x0.5 with 98.06%, followed by ShuffleNetV2 x1.0 with 97.66%, ShuffleNetV2 x1.5 with 97.21%. These top three models are considered as high accuracy models as there is only a slight difference in their accuracy. Following that, ShuffleNetV2 x2.0 with 95.62% and VGG16 with 95.84% respectively, are the average performed models in terms of classification accuracy. Meanwhile, MobileNetV2 with 91.02%, MobileNetV3 - Small with 87.40%, and MobileNetV3 - Large with 82.88%, obtained lower classification accuracy.

Based on the accuracy analysis, between the ShuffleNetV2 variants to VGG16 model, the classification accuracy is dropping, while VGG16 may have a better performance on the training output. This is due to its larger network (i.e., 138M number of parameters) compared to ShuffleNetV2 that has a smaller network (i.e., x0.5 with 1.4M, x1.0 with 2.3M, x1.5 with 3.6M, and 2.0 with 7.4M), VGG16 must have too many redundant parameters that cause it to lose inference accuracy. Moreover, comparing MobileNetV2 and MobileNetV3 with ShuffleNetV2, the lower train and validation accuracy cause the lower inference accuracy. This suggests that MobileNetV2 and

MobileNetV3 need to further increase the training epoch or fine-tune their hyperparameter settings to increase their classification accuracy.

TABLE II. TRAINING AND VALIDATION DATA FOR INITIAL AND FINAL

Model	Epoch	Train Loss (%)	Train Accuracy (%)	Val. Loss (%)	Val. Accuracy (%)
MobileNetV2	Initial	2.15	24.33	1.68	23.42
	Final	0.54	80.26	0.45	83.17
MobileNetV3-Small	Initial	2.16	24.94	1.81	25.63
	Final	0.67	74.95	0.63	78.77
MobileNetV3-Large	Initial	2.81	19.29	1.95	17.91
	Final	0.73	72.84	0.59	76.14
ShuffleNetV2x0.5	Initial	0.96	64.60	0.70	75.48
	Final	0.23	92.08	0.51	84.53
ShuffleNetV2x1.0	Initial	0.91	67.12	0.94	70.54
	Final	0.20	93.20	0.61	85.44
ShuffleNetV2x1.5	Initial	1.43	42.46	1.13	52.82
	Final	0.19	93.48	0.49	85.77
ShuffleNetV2x2.0	Initial	1.51	38.90	1.20	45.77
	Final	0.18	93.67	0.56	85.86
VGG-16	Initial	1.39	42.36	1.13	55.11
	Final	0.11	96.25	0.28	89.37

TABLE III. CLASSIFICATION ACCURACY FROM INFERENCE

Model	Accuracy (%)	Inf. Time (ms)	FPS
Mobilenetv2	91.02	14.50	68.98
Mobilenetv3Small	87.40	10.63	94.08
Mobilenetv3Large	82.88	14.15	70.70
Shufflenetv2:X0.5	98.06	9.41	106.76
Shufflenetv2:X1.0	97.66	12.30	81.33
Shufflenetv2:X1.5	97.21	14.80	67.84
Shufflenetv2:X2.0	95.62	17.12	58.44
VGG16	95.84	82.40	12.22

Next, in terms of inference speed, ShuffleNetV2 x0.5 with 9.41 ms and 106.76 FPS provide faster inference speed

compared to the other models. Meanwhile VGG16 has the slowest inference time with 82.40 ms and 12.22 FPS. This is basically related to the model's number of parameters which reflect the timing output of the inferences. Other ShuffleNetV2, MobileNetV2, and MobileNetV3 also have quite a lower inference time and higher FPS compared to VGG16 that is very slow.

B. Performance comparison

For more detailed analysis, Table IV shows some prior works that used the same selected models for their edge application. First, based on MobileNetV2, researcher [18] have achieved 96.87%, researcher [19] have achieved 96.52%, and researcher [20] have achieved 94.76%, respectively in their classification accuracy. Compared to the MobileNetV2 used in this study, that is only able to achieve 91.02% in classification accuracy, their model has been further optimized to increase the performance based on their edge devices. In contrast, MobileNetV3 in [21] have lower FPS compared to the MobileNetV3 in this study, that able to achieve 94.08 FPS for MobileNetV3-Small and 70.70 FPS for MobileNetV3-Large. This suggests that, while the model optimization that has been done in their work might not significantly increase the model number of parameters, different edge devices have a different processing power that may cause this inconsistency. Furthermore, for ShuffleNetV2 in [18], [22], the accuracy obtained are 99% and 99.58% respectively. In contrast to the ShuffleNetV2 models in this study, their ShuffleNetV2 have really high accuracy. Besides that, the VGG16 in [23] with 95.3% has a quite close to the accuracy of VGG16 obtained in this study, while compared to research in [24] the VGG16 accuracy was 55% with 5 FPS.

TABLE IV. PRIOR WORKS OF SELECTED MODELS INFERENCE ON EDGE DEVICES

Model	Paper	Accuracy (%)	FPS
Mobilenetv2	[18]	96.87	-
Mobilenetv2	[19]	96.52	-
Mobilenetv2	[20]	94.76	-
Mobilenetv3	[21]	-	30
Shufflenetv2	[18]	99	-
Shufflenetv2	[22]	99.58	-
VGG16	[23]	95.3	-
VGG16	[24]	55	5

To summarize, for ShuffleNetV2, a high train accuracy and val accuracy can be obtained even with small number of training epoch compared to MobileNetV2 and MobileNetV3. In terms of inference, ShuffleNetV2 can obtain higher classification accuracy with a smaller network compared to VGG16. For speed, ShuffleNetV2 also has faster inference time and FPS compared to the others. In addition, prior works also

shows that ShuffleNetV2 has high performance in accuracy compared to MobileNetV2, MobileNetV3, and VGG16 for edge applications.

IV. CONCLUSION

In conclusion, the analysis of several lightweight CNN models tailored specifically for deployment on an edge device for image classification tasks have been successfully achieved. This work was done by conducting a thorough investigation to understand a few recently produced lightweight CNN models for image classification task on edge devices. With that, some of the most-used models (i.e., MobileNet, ShuffleNet, and VGG16) were selected for performance evaluation. In order to conduct the performance evaluation, a transfer learning approach were used on those models. In addition, the training and validation were done on a computer and their inference were deployed and ran on Jetson Nano. The result suggested that the most consistent performance across training, validation, and inference was produced by ShuffleNetV2 with the highest 98.06% of accuracy, faster inference time with 9.41 ms and 106.76 FPS compared to the other models. Overall, based on the analysis on several models, generally ShuffleNetV2 is the most feasible model to be deployed on Jetson Nano.

ACKNOWLEDGMENT

This research article was financially supported by University Teknologi MARA and Institute of Postgraduate Studies UiTM. The authors would like to sincerely thank Universiti Teknologi MARA Cawangan Pulau Pinang for their invaluable support in this research, providing access to essential resources and materials. We are also deeply grateful to our fellow researchers, especially the dedicated members of the Machine Learning Research Group (MLRG) at the Centre for Electrical Engineering Studies, whose collaboration and insights were crucial in achieving the best possible outcomes for this study.

REFERENCES

- [1] R. Y. Choi, A. S. Coyner, J. Kalpathy-Cramer, M. F. Chiang, and J. Peter Campbell, "Introduction to machine learning, neural networks, and deep learning," *Transl Vis Sci Technol*, vol. 9, no. 2, 2020, doi: 10.1167/tvst.9.2.14.
- [2] N. Sharma, R. Sharma, and N. Jindal, "Machine Learning and Deep Learning Applications-A Vision," *Global Transitions Proceedings*, vol. 2, no. 1, pp. 24–28, Jun. 2021, doi: 10.1016/j.gltp.2021.01.004.
- [3] R. Yamashita, M. Nishio, R. K. G. Do, and K. Togashi, "Convolutional neural networks: an overview and application in radiology," Aug. 01, 2018, *Springer Verlag*. doi: 10.1007/s13244-018-0639-9.
- [4] X. Hou, Y. Guan, T. Han, and N. Zhang, "DistrEdge: Speeding up Convolutional Neural Network Inference on Distributed Edge Devices," Feb. 2022, [Online]. Available: <http://arxiv.org/abs/2202.01699>
- [5] S. Naveen, M. R. Kounte, and M. R. Ahmed, "Low Latency Deep Learning Inference Model for Distributed Intelligent IoT Edge Clusters," *IEEE Access*, vol. 9, pp. 160607–160621, 2021, doi: 10.1109/ACCESS.2021.3131396.
- [6] R. Stahl, A. Hoffman, D. Mueller-Gritschneider, A. Gerstlauer, and U. Schlichtmann, "DeeperThings: Fully Distributed CNN Inference on Resource-Constrained Edge Devices," *Int J Parallel Program*, vol. 49, no. 4, pp. 600–624, Aug. 2021, doi: 10.1007/s10766-021-00712-3.
- [7] S. P. Baller, A. Jindal, M. Chadha, and M. Gerndt, "DeepEdgeBench: Benchmarking Deep Neural Networks on Edge Devices," Aug. 2021, [Online]. Available: <http://arxiv.org/abs/2108.09457>
- [8] J. Liu, S. Tripathi, U. Kurup, and M. Shah, "Pruning Algorithms to Accelerate Convolutional Neural Networks for Edge Applications: A Survey," May 2020, [Online]. Available: <http://arxiv.org/abs/2005.04275>
- [9] F. Wang, M. Zhang, X. Wang, X. Ma, and J. Liu, "Deep Learning for Edge Computing Applications: A State-of-the-Art Survey," *IEEE Access*, vol. 8, pp. 58322–58336, 2020, doi: 10.1109/ACCESS.2020.2982411.
- [10] S. S. A. Zaidi, M. S. Ansari, A. Aslam, N. Kanwal, M. Asghar, and B. Lee, "A survey of modern deep learning based object detection models," Jun. 30, 2022, *Elsevier Inc.* doi: 10.1016/j.dsp.2022.103514.
- [11] V. Kamath and A. Renuka, "Deep learning based object detection for resource constrained devices: Systematic review, future trends and challenges ahead," Apr. 28, 2023, *Elsevier B.V.* doi: 10.1016/j.neucom.2023.02.006.
- [12] H. Hussain, P. S. Tamizharasan, and C. S. Rahul, "Design possibilities and challenges of DNN models: a review on the perspective of end devices," *Artif Intell Rev*, vol. 55, no. 7, pp. 5109–5167, Oct. 2022, doi: 10.1007/s10462-022-10138-z.
- [13] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," Jan. 2018, [Online]. Available: <http://arxiv.org/abs/1801.04381>
- [14] A. Howard *et al.*, "Searching for MobileNetV3," May 2019, [Online]. Available: <http://arxiv.org/abs/1905.02244>
- [15] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," Jul. 2018, [Online]. Available: <http://arxiv.org/abs/1807.11164>
- [16] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," Sep. 2014, [Online]. Available: <http://arxiv.org/abs/1409.1556>
- [17] A. Manvikar and P. Reddy, "Dataset of groundnut plant leaf images for classification and detection," 2023. doi: <http://doi.org/10.17632/22p2vcxbxfk.3>.
- [18] J. Bonam, S. S. Kondapalli, N. L. V. Prasad, and K. Marlapalli, "Lightweight CNN Models for Product Defect Detection with Edge Computing in Manufacturing Industries," *J Sci Ind Res (India)*, vol. 82, no. 4, pp. 418–425, Apr. 2023, doi: 10.56042/jsir.v82i04.72390.
- [19] L. R. Burra, A. Karuna, S. Tumma, K. Marlapalli, and P. Tumuluru, "MobileNetV2-based Transfer Learning Model with Edge Computing for Automatic Fabric Defect Detection," *J Sci Ind Res (India)*, vol. 82, no. 1, pp. 128–134, Jan. 2023, doi: 10.56042/jsir.v82i1.69928.
- [20] T. Yoo, S. Lee, and T. Kim, "Dual image-based cnn ensemble model for waste classification in reverse vending machine," *Applied Sciences (Switzerland)*, vol. 11, no. 22, Nov. 2021, doi: 10.3390/app112211051.
- [21] B. Wang, F. Ma, L. Ge, H. Ma, H. Wang, and M. A. Mohamed, "Icing-EdgeNet: A Pruning Lightweight Edge Intelligent Method of Discriminative Driving Channel for Ice Thickness of Transmission Lines," *IEEE Trans Instrum Meas*, vol. 70, 2021, doi: 10.1109/TIM.2020.3018831.
- [22] L. V. Narasimha Prasad, D. B. Dokku, S. L. Talasila, and P. Tumuluru, "Hyper Parameter Optimization for Transfer Learning of ShuffleNetV2 with Edge Computing for Casting Defect Detection," *J Sci Ind Res (India)*, vol. 82, no. 2, pp. 171–177, 2023, doi: 10.56042/jsir.v82i2.70250.
- [23] S. Chang and B. Zheng, "A lightweight convolutional neural network for automated crack inspection," *Constr Build Mater*, vol. 416, Feb. 2024, doi: 10.1016/j.conbuildmat.2024.135151.
- [24] E. Kristiani, Y. T. Tsan, P. Y. Liu, N. Y. Yen, and C. T. Yang, "Binary and Multi-Class Assessment of Face Mask Classification on Edge AI Using CNN and Transfer Learning," *Human-centric Computing and Information Sciences*, vol. 12, 2022, doi: 10.22967/HGIS.2022.12.053.